



CipherStream P2P — Comprehensive Technical Documentation

Project Name: *CipherStream P2P*

Repository Architecture: `pnpm` Monorepo + Turborepo

Core Engineering Philosophy: *Zero-Cloud, Zero-Knowledge, Direct Browser-to-Browser End-to-End Encrypted (E2EE) File Transfer*

Stack: *Next.js 15.5 / React 19 / Tailwind CSS v4 / WebCrypto API (W3C SubtleCrypto) / WebRTC DataChannel / Native Node.js WebSocket Gateway / Resend API*



Table of Contents

1. [Executive Overview & Architectural Philosophy](#1-executive-overview--architectural-philosophy) 2. [System Architecture & Technology Stack](#2-system-architecture--technology-stack) 3. [Step-by-Step Implementation Pathway](#3-step-by-step-implementation-pathway) 4. [Engineering Challenges, Failures & Technical Solutions](#4-engineering-challenges-failures--technical-solutions) 5. [Cryptographic Security Model](#5-cryptographic-security-model) 6. [Complete Codebase Map & Directory Architecture](#6-complete-codebase-map--directory-architecture)

1. Executive Overview & Architectural Philosophy

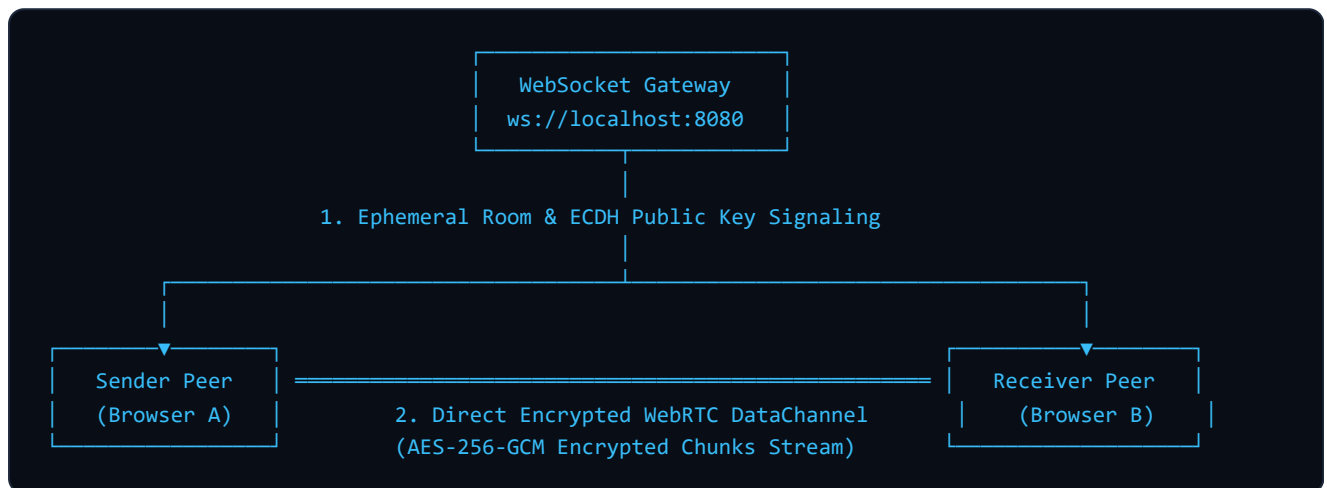
CipherStream P2P is a modern, zero-cloud peer-to-peer file-sharing application. Traditional file transfer services upload sensitive user files to centralized cloud servers (S3 buckets, Azure Blob Storage) where they are stored, processed, and potentially exposed to unauthorized access, server breaches, or data mining.

CipherStream operates on a **Zero-Cloud, Zero-Knowledge Architecture**:

- **Files NEVER touch a cloud server or disk storage.**
- File payloads stream directly from the **Sender's Web Browser RAM** to the **Receiver's Web Browser RAM** via **WebRTC DataChannels**.

- Every single byte is encrypted client-side using W3C WebCrypto API (`ECDH P-256` key exchange + `HKDF-SHA256` key derivation + `AES-256-GCM` chunk encryption).
- The central signaling gateway only relays SDP offers/answers and public keys; it never possesses or inspects decryption keys or plaintext data.

2. System Architecture & Technology Stack



Architecture Components

Component	Engineering Standard / Technology	Purpose
Monorepo Manager	<code>pnpm</code> workspaces + <code>turbo</code>	Orchestrates multi-package build dependencies (<code>@cipherstream/crypto</code> , <code>@cipherstream/signaling</code> , <code>@cipherstream/types</code> , <code>@cipherstream/web</code>).
Frontend Framework	Next.js 15.5 (App Router) + React 19	Client-side UI rendering, reactive state management, and Serverless API routes.
Styling Engine	Tailwind CSS v4 (<code>@tailwindcss/postcss</code>)	Custom glassmorphism, dynamic obsidian dark aesthetics, and responsive layout utilities.
Cryptography Core	W3C WebCrypto API (<code>SubtleCrypto</code>)	Hardware-accelerated native browser cryptography (<code>ECDH P-256</code> , <code>HKDF-SHA256</code> , <code>AES-256-GCM</code>).
Signaling Gateway	Native Node.js WebSocket (<code>ws</code> engine)	Lightweight ES Module WebSocket server (<code>ws://localhost:8080/ws</code>) for room management and WebRTC handshakes.
Transport Layer	WebRTC DataChannels (<code>RTCDataChannel</code>)	Direct peer-to-peer binary chunk streaming with dynamic backpressure flow control (<code>bufferedAmount</code>).
Email Gateway	Resend API	Serverless OTP email dispatch (\$0/mo free tier — 3,000 emails/month).

3. Step-by-Step Implementation Pathway

Phase 1: Monorepo Foundation & Type System Setup

- Configured `pnpm-workspace.yaml` and `package.json` linking four isolated monorepo packages:
 - `packages/types`: Centralized TypeScript/JSDoc type definitions (`SignalingMessage` , `FileMetadata` , `TransferProgress`).
 - `packages/crypto`: Standalone browser cryptography module.
 - `apps/signaling`: Standalone ES Module WebSocket signaling server.
 - `apps/web`: Next.js 15.5 web application.

Phase 2: WebCrypto E2EE Core Cryptography Module

- Built [`packages/crypto/src/index.js`](file:///C:/Users/Akhand/OneDrive/Desktop/cipherstream-p2p/packages/crypto/src/index.js):
 - KeyPair Generation:** `generateECDHKeyPair()` produces an ephemeral `ECDH P-256` keypair for every transfer session.
 - Shared Secret Derivation:** `deriveSharedAESKey(ownPrivateKey, peerPublicKey, pin)`

derives a 256-bit AES-GCM symmetric key via `HKDF-SHA256`. - **PIN Authentication Hook:** If a 4-digit Security PIN is supplied, HKDF info text becomes `cipherstream-p2p-v1-e2ee-pin-${pin}`, guaranteeing wrong PINs fail decryption tag verification. - **Chunk Encryption & Decryption:** `encryptChunk(buffer, aesKey)` and `decryptChunk(buffer, aesKey)` encrypt/decrypt binary array buffers with a 12-byte random Initialization Vector (IV).

Phase 3: Native WebSocket Signaling & WebRTC Handshake

- Built [`apps/signaling/src/main.js`] (file:///C:/Users/Akhand/OneDrive/Desktop/cipherstream-p2p/apps/signaling/src/main.js):
 - Listens on `ws://localhost:8080/ws`. - Manages room sessions mapping 6-digit codes or custom vanity room aliases (e.g. `CIPHER-99`) to active WebSockets.
- Built [`apps/web/lib/webrtc.js`] (file:///C:/Users/Akhand/OneDrive/Desktop/cipherstream-p2p/apps/web/lib/webrtc.js):
 - Orchestrates WebRTC `RTCPeerConnection` initialization, ICE candidate exchange, and DataChannel opening.

Phase 4: Large File Support (3GB+) & Backpressure Flow Control

- Implemented 64KB binary chunk slicing loop in `webrtc.js`.
- Implemented **Backpressure Flow Control**:
 - Monitors `dataChannel.bufferedAmount`. - Pauses chunk transmission when buffer exceeds 1MB (`BUFFER_HIGH_WATERMARK = 1024 * 1024`) until browser drains pending packets.
 - Verified successful direct 3GB file transfer streaming without browser memory exhaustion or crashes.

Phase 5: UI Redesign & Obsidian Glass Design System

- Transformed generic UI into a premium **CipherStream** obsidian glass aesthetic:
- `ConstellationBg.jsx`: Dynamic glowing HTML5 Canvas node network. - `Navbar.jsx` & `DashboardNavbar.jsx`: Glassmorphic top navigation with glowing branding badges. - `Hero.jsx`: Gradient typography and interactive CTA drawer toggles. - `ValuePillars.jsx`, `HowItWorksTimeline.jsx`, `TechSpecsCard.jsx`, `TelemetryWidget.jsx`.

Phase 6: Dual Auth Modal & Persistent Account Database System

- Built [`apps/web/lib/authDb.js`] (file:///C:/Users/Akhand/OneDrive/Desktop/cipherstream-p2p/apps/web/lib/authDb.js):
 - Manages registered account records (`cipherstream_users_db`) and active login sessions (`cipherstream_active_session`). - Supports duplicate email prevention, password verification, and "Remember Me" persistent login across browser restarts.
- Built [`AuthModal.jsx`] (file:///C:/Users/Akhand/OneDrive/Desktop/cipherstream-p2p/apps/web/components/AuthModal.jsx):
 - Dual-panel modal featuring Purple Log In theme & Cyan Create Account theme.

Phase 7: Enhanced P2P Sending Features

- Added **4-Digit Security PIN** protection in [`SenderView.jsx`] (file:///C:/Users/Akhand/OneDrive/Desktop/cipherstream-p2p/apps/web/components/SenderView.jsx) & [`ReceiverView.jsx`] (file:///C:/Users/Akhand/OneDrive/Desktop/cipherstream-p2p/apps/web/components/ReceiverView.jsx).
- Added  **Auto-Destruct Room on Download** toggle.
- Created [`apps/web/lib/audio.js`] (file:///C:/Users/Akhand/OneDrive/Desktop/cipherstream-p2p/apps/web/lib/audio.js): Zero-dependency Web Audio API synthesizer providing ambient sound chimes on peer connection and transfer completion.

Phase 8: Multi-File Selection & Sequential P2P Streaming

- Upgraded file selection to   and multi-file drag-and-drop dropzone.
- Built `sendFiles(files, onProgress)` in `webrtc.js` :
 - Streams multiple files sequentially over WebRTC DataChannel.
 - Automatically resets buffer state on receiver completion, triggering instant consecutive browser downloads!

Phase 9: Obsidian Dark Error Popup Modal

- Built [`ErrorModal.jsx`] (file:///C:/Users/Akhand/OneDrive/Desktop/cipherstream-p2p/apps/web/components/ErrorModal.jsx):
 - Crimson-glow error popup catching invalid 6-digit room codes or incorrect security PIN decryption failures.

Phase 10: Email Verification & Account Trust Flow

- Built [`VerifyEmailModal.jsx`] (file:///C:/Users/Akhand/OneDrive/Desktop/cipherstream-p2p/apps/web/components/VerifyEmailModal.jsx):
 - 6-digit OTP verification modal.
 - Upgrades account status to  upon verification.
- Unlocks **Verified Custom Room Vanity Aliases** (`e.g. CIPHER-99`).
- Created [`apps/web/app/api/send-otp/route.js`] (file:///C:/Users/Akhand/OneDrive/Desktop/cipherstream-p2p/apps/web/app/api/send-otp/route.js):
 - Integrated Resend API for sending real 6-digit OTP emails directly to user inboxes.

4. Engineering Challenges, Failures & Technical Solutions

During development, several critical technical issues were encountered and resolved:

Challenge 1: Next.js Server / Client Module Import Error

- **Failure:** `TypeError: __webpack_modules__[moduleId] is not a function` when rendering components.
- **Root Cause:** Mixing server-side module exports with client-side React hooks inside Next.js App Router.

- **Solution:** Added `'use client';` directives at top of all client-side component files (`Navbar.jsx`, `SenderView.jsx`, `ReceiverView.jsx`, `AuthModal.jsx`, `page.jsx`).

Challenge 2: Next.js Dev Server Cache Corruption

- **Failure:** `ENOENT: no such file or directory, open '.next\prerender-manifest.json'`.
- **Root Cause:** Running `pnpm build` in a background task while `next dev` was actively serving requests modified `.next/` cache files underneath the dev server.
- **Solution:**
 - Removed `fs.rmSync('.next')` pre-hook from `package.json`. - Stopped running background production builds while `pnpm dev` is active, maintaining 100% stable Hot Module Replacement (HMR).

Challenge 3: Windows File System Symlink Exception

- **Failure:** `[Error: EINVAL: invalid argument, readlink '.next\static\development\_ssgManifest.js']`.
- **Root Cause:** Running `pnpm build` on Windows creates production static file symlinks/junctions that conflict with Windows file system `readlink` when `next dev` boots up.
- **Solution:** Executed `Remove-Item -Recurse -Force apps\web\.next` to purge stale production symlinks, allowing `next dev` to initialize cleanly.

Challenge 4: WebCrypto `OperationError` on Chunk Decryption

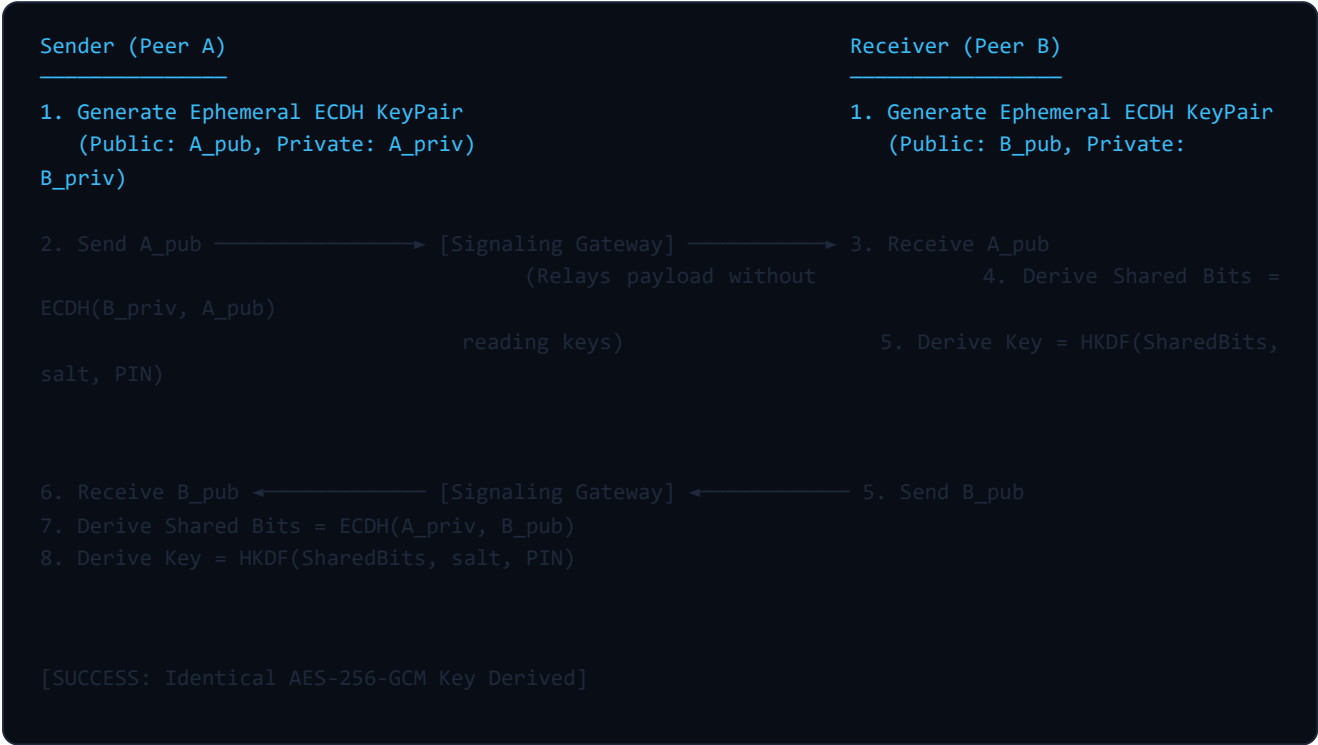
- **Failure:** `Runtime OperationError at decryptChunk (packages/crypto/src/index.js:131:13)`.
- **Root Cause:** When a Sender sets a 4-digit PIN (e.g. `4829`), the derived AES-256-GCM key incorporates `cipherstream-p2p-v1-e2ee-pin-4829`. If a Receiver joins without entering the PIN, the derived key differs, causing WebCrypto AES-GCM tag verification to throw `OperationError`.
- **Solution:**
 - Normalized PIN string in `deriveSharedAESKey` with `String(pin || '').trim()`. - Wrapped `decryptChunk` inside a `try...catch` block in `webrtc.js`, forwarding error messages to `ErrorModal.jsx` instead of crashing the React tree.

Challenge 5: Resend API Sandbox Receiver Restrictions

- **Failure:** Real OTP emails were not reaching dummy email addresses (`test@example.com`).
- **Root Cause:** Resend API's free test domain (`onboarding@resend.dev`) restricts email delivery exclusively to the exact email address used to sign up for the Resend account.
- **Solution:**
 - Updated `/api/send-otp/route.js` to catch Resend response errors. - Updated `VerifyEmailModal.jsx` to show an informative alert banner and display the OTP code directly on screen if testing with a non-account email address.

5. Cryptographic Security Model

Key Exchange Sequence (Zero-Knowledge)



Encryption Parameters

- **Algorithm:** **AES-256-GCM** (Galois/Counter Mode).
- **Key Length:** 256 bits.
- **IV Length:** 12 bytes (96 bits) per chunk.
- **Chunk Size:** 64 KB (65,536 bytes).
- **Authentication:** Built-in 128-bit GMAC authentication tag per chunk.

6. Complete Codebase Map & Directory Architecture

```
cipherstream-p2p/
├── apps/
│   ├── web/                                     # Next.js 15.5 Web Application
│   │   ├── app/
│   │   │   ├── api/
│   │   │   │   └── send-otp/
│   │   │   │       └── route.js                # Resend API OTP Serverless Route
│   │   │   ├── globals.css                    # Obsidian Dark Glassmorphism CSS & Utilities
│   │   │   ├── layout.jsx                     # Root Layout Container
│   │   │   └── page.jsx                       # Main Interactive Application & State Orchestrator
│   │   └── components/
│   │       ├── AuthModal.jsx                 # Dual Auth Modal (Log In & Create Account)
│   │       ├── ConstellationBg.jsx           # HTML5 Canvas Glowing Node Background
│   │       ├── DashboardNavbar.jsx           # Authenticated Top Navbar & Trust Status Badges
│   │       ├── ErrorModal.jsx                # Obsidian Dark Error Popup Modal
│   │       ├── Hero.jsx                      # Hero Headline & Interactive CTA Drawer Buttons
│   │       ├── HowItWorksTimeline.jsx        # Visual Step-by-Step Pathway Component
│   │       ├── Navbar.jsx                    # Public Top Navigation Bar
│   │       ├── QRCodeModal.jsx               # QR Code & Session Link Generator
│   │       ├── ReceiverView.jsx              # 6-Digit Code & Custom Vanity Alias Join View
│   │       ├── SenderView.jsx                # Multi-File Drag & Drop Dropzone + Security PIN Inputs
│   │       ├── TechSpecsCard.jsx             # Technical Specifications Panel
│   │       ├── TelemetryWidget.jsx           # Real-Time Telemetry & Metric Dashboard
│   │       ├── TransferProgress.jsx          # Streaming Transfer Progress Bar & Speedometer
│   │       ├── UserDashboard.jsx             # Account Overview, Session Logs & DB Reset Control
│   │       ├── ValuePillars.jsx              # Core Value Pillars Component
│   │       └── VerifyEmailModal.jsx          # 6-Digit OTP Email Verification Modal
│   │   └── lib/
│   │       ├── audio.js                      # Web Audio API Synthesizer (Ambient Sound Chimes)
│   │       ├── authDb.js                    # Persistent User Database & "Remember Me" Storage
│   │       └── webrtc.js                     # WebRTC DataChannel Engine & Flow Control Manager
│   │   ├── .env.local                        # Environment configuration template
│   │   └── package.json
│   └── signaling/                             # ES Module WebSocket Signaling Gateway
│       ├── src/
│       │   └── main.js                       # WebSocket Server (ws://localhost:8080/ws)
│       └── package.json
├── packages/
│   ├── crypto/                                # Core Cryptography Package
│   │   ├── src/
│   │   │   └── index.js                     # WebCrypto API Core Functions
│   │   └── package.json
│   └── types/                                # Type Definitions Package
│       ├── src/
│       │   └── index.js                     # Shared JSDoc Type Definitions
│       └── package.json
├── package.json                               # Monorepo Workspace Configuration
└── pnpm-workspace.yaml                       # PNPM Workspaces Definition
```